

Homework 16: A Log Tool

Build something that reads a file properly, writes one safely, and refuses a path traversal attempt.

Every file you create must be deleted before the program exits. Work inside

`Files.createTempDirectory(...)` and clean it up in a `finally`. Leaving files behind fails this assignment.

Part One: Stream, Do Not Slurp

Create `LogTool.java` in the classic form.

Generate a log of 1,000 lines shaped like:

```
ERROR request=0 ms=10
INFO request=1 ms=47
```

Then answer four questions, **each with its own** `Files.lines` **call**:

1. How many lines of each level, in alphabetical order of level
2. The first `ERROR` line
3. The line with the highest `ms=` value
4. Summary statistics over all the `ms=` values

Every one of those streams must be closed. Write a comment saying what would leak if you omitted the `try-with-resources`, and how many times over in this method.

For question 2, add a comment on what `findFirst` does that `readAllLines` could never do, whatever the file size.

Part Two: Write A Filtered Copy

Write only the `ERROR` lines to a second file, reading and writing **in one try-with-resources with two resources**.

You will hit a problem: `forEach` takes a `Consumer`, and `write` throws a checked `IOException`. Solve it, and write a comment naming the general problem and why the standard fix preserves the original failure.

Then:

- Append a trailing line, and comment on why the append needed an explicit option
- Create a nested directory and `move` the file into it
- Show the original is gone

Part Three: The Path Validator

Create an `uploads` directory. Write a method that takes a user-supplied filename and decides whether it is allowed.

Test it with all five of these:

Input	Should be
<code>photo.jpg</code>	allowed
<code>nested/../photo.jpg</code>	allowed
<code>../../etc/passwd</code>	rejected
<code>/etc/passwd</code>	rejected
<code>..</code>	rejected

The fourth one is the interesting case. Work out what `resolve` does with an absolute argument, and make sure your check still catches it.

Then Prove The String Trap

Create a sibling directory called `uploads-evil`, and a file inside it.

Print whether that file is "inside uploads" according to `String.startsWith` and according to `Path.startsWith`. They disagree.

Write a comment explaining why, what `Path` compares that `String` does not, and why this is a real bypass rather than a curiosity.

Part Four: Exceptions That Say Something

Trigger and catch each of these, printing the filename from the exception itself rather than from a variable you already had:

1. `NoSuchFileException`
2. `FileAlreadyExistsException`
3. `DirectoryNotEmptyException`

Then call `delete()` on a missing file through the **legacy** `java.io.File` API and print what it returns.

Write a comment covering:

- What the legacy return value tells you, and what it cannot
- **Why checking `exists()` first is not a fix.** Name the race and say why the exception can still arrive from a line that looks guarded

Acceptance Criteria

- ☐ Runs with `java LogTool.java`
- ☐ The temp directory is deleted, and the program prints proof
- ☐ Four separate `Files.lines` calls, each in try-with-resources
- ☐ A comment states what leaks without them, and how many times
- ☐ A comment explains what `findFirst` gains over `readAllLines`
- ☐ The filtered copy uses two resources in one try-with-resources
- ☐ The checked-exception-in-a-lambda problem is solved and explained
- ☐ The append uses an explicit open option, with a comment on why
- ☐ All five validator cases produce the correct verdict
- ☐ `String.startsWith` and `Path.startsWith` are shown disagreeing
- ☐ The explanation says what `Path` compares instead
- ☐ All three file exceptions are caught and their filename printed from the exception
- ☐ A comment names the TOCTOU race and why `exists()` does not fix it

Stretch

Make the filtered copy **atomic**: write to a temporary file in the same directory, then move it into place with `StandardCopyOption.ATOMIC_MOVE`.

Then write a comment on what a reader of the destination file could observe during a non-atomic write, and why "same directory" matters for the move to be atomic at all.

Hint, if `/etc/passwd` slips through

Check what `base.resolve("/etc/passwd")` returns before you normalise. An absolute argument does not append to the base, it replaces it entirely, so the result has nothing to do with your uploads directory. Your containment check is what has to catch that, which is exactly why the check exists rather than trusting `resolve`.